Local vs. Global Variables & Scope

Module 1: Python Foundations

```
def make_total():
    total = 100
make_total()
print(total)
```

By the End of Today, You'll Be Able To…

1

Explain local vs. global variables

2

Explain why globals make code harder to reuse

CHECK FOR UNDERSTANDING

Given `double(n)`: `return` n × 2, what does `double(5)` return?

A

5

B

n × 2

C

10

ANSWER KEY

Prior-Lesson Function Recall

Correct answer: 10

`n` becomes 5 for this call, `n * 2` becomes 10, and `return` hands that value back.

What Happens In Between?

Parameters get values in and `return` sends a value out — but where can that variable actually be used while the function runs?

A Local Variable Only Exists Inside Its Function

CREATE

Created when the function runs.

A Local Variable Only Exists Inside Its Function

CREATE

Created when the function runs.

USE

Visible only inside that function.

A Local Variable Only Exists Inside Its Function

CREATE

Created when the function runs.

USE

Visible only inside that function.

GONE

Stops existing when the function ends.

Like a sticky note on your own desk: useful while you're there, not shared with every desk.

Something Different: Global

Not every variable disappears when a function ends. A variable created outside any function works differently.

A Global Variable Exists for the Whole Program

CREATE

Created outside any function.

READ

Readable from inside functions.

STAYS

Exists for the whole program.

Like a classroom whiteboard: everyone can see it, and it stays up for the whole class.

Local Scope vs. Global Scope

LOCAL

- Created inside a function
- Visible only there
- Gone when the function ends

GLOBAL

- Created outside functions
- Readable inside functions
- Exists for the whole program

REAL-WORLD SCENARIO

A Grant Budget Used the Wrong Overhead Rate

```
rate = 0.10    # cell 2

def grant_total(amount):
    return amount * (1 + rate)

rate = 0.15    # cell 40
grant_total(50000)  # which rate?
```

Illustrative code for the scenario

A function read a global overhead rate
A teammate changed that global later
Every later total used the new rate

The function call never showed which rate it used. How would you make that visible?

Illustrative scenario, not a real event

Why Not Just Use Globals for Everything?

If a global is readable from anywhere, why bother with parameters? This is the actual point of today.

Globals Make a Function Harder to Reuse

PARAMETERS

Show what the function needs.

GLOBALS

Hide dependencies from the call.

TESTING

Needs the global set up first.

CODE ALONG

Open the Code-Along

Open `Module1_Lesson4_local-vs-global-variables-scope_STUDENT.py`
Run it after every section: `python3` + the file name

Sections, in order:

1. Local Scope, Live · see the real error once it's gone
2. Reading a Global, Live · see what that convenience hides
3. Refactor: Global to Parameter · make the hidden input visible

Local Scope, Live

python

```python
def make_total():
    total = 100
    return total

make_total()
print(total)
```

Reading a Global, Live

python

```python
tax_rate = 0.08

def price_with_tax(price):
    return price + (price * tax_rate)

print(price_with_tax(100))
```

Refactor: Global to Parameter

```python
tax_rate = 0.08

def price_with_tax(price, tax_rate):
    return price + (price * tax_rate)

print(price_with_tax(100, tax_rate))
```

CODE-ALONG DONE

Back to the Slides: Review

You just typed it and ran it. Now check that it stuck, without the code in front of you.

Watch It Run: A Local Variable Disappears

Step 1 of 5

Python learns make_total. Nothing runs yet

```
def make_total():
    total = 100
    return total
make_total()
print(total)
```

OUTPUT

ANIMATED IN CLASS: watch it run in the lecture recording

ADD TO THE BOARD

Put the Local Variable Lifecycle in Order

A

`total` stops existing

B

`make_total()` is called

C

`total` is created and set to 100

```
def make_total():
    total = 100
    return total
make_total()
```

QUICK CHALLENGE

`set_x()` runs and sets `x = 5`, but `print(x)` outside raises `NameError`. Why?

A `set_x()` didn't `return x`

B `x` only existed inside `set_x()`

C `print()` only works on strings

ANSWER KEY

Local Scope Trace

Correct: `x` only existed inside `set_x()`

`x` was local, so it was gone once `set_x()` ended. Returning `x` only helps if the caller stores it: `x = set_x()`

QUICK CHALLENGE

A function works here but breaks when copied elsewhere. Most likely cause?

A It secretly depends on a missing global

B Functions can’t be copied

C Its parameter names are wrong

ANSWER KEY

Global Dependency Debug

Correct: It secretly depends on a missing global

A function that reads a global silently depends on that name existing wherever it runs. Copy the function without the global, and it breaks.

TIP

“Why Avoid Globals Inside Functions?”

GLOBAL

```
price_with_tax(100)
```

PARAMETER

```
price_with_tax(100, tax_rate)
```

Answer like an interview, with PREP: point, reason, example, point.

MODEL ANSWER

Model Answer: PREP

POINT

I pass values in as parameters instead of reading globals.

REASON

A global is a hidden dependency the call doesn't show.

EXAMPLE

`price_with_tax(price)` silently needed `tax_rate` from outside.

POINT, RESTATED

Parameters make a function testable and safe to reuse.

CHECK FOR UNDERSTANDING

`count = 0` globally; `add()` assigns `count = count + 1` locally. What happens?

A Returns 1

B Returns 0

C Raises an error

ANSWER KEY

Scope Synthesis Trace

Correct answer: Raises an error

Assigning to `count` inside `add()` makes Python treat it as local. `count + 1` then tries to read that local value before it has been assigned.

So Far, We've Covered…

BEFORE

Functions and parameters

TODAY

Local scope disappears; global scope doesn't

NEXT

Retrieval practice on everything so far

KEEP LEARNING

Python Tutor — Visualize Variable Scope

PRACTICE • 5–10 MIN

Watch a local variable disappear in real time.

Open Python Tutor

Python Docs — Scope